

Pilha com Operações `min()` e `max()` em $O(1)$

Análise de Complexidade de Tempo e Espaço

1. Introdução

Uma pilha (stack) é uma estrutura de dados linear que segue a política LIFO (Last In, First Out). As operações fundamentais são push (inserção no topo) e pop (remoção do topo), ambas executadas em tempo $O(1)$.

O desafio proposto consiste em estender essa estrutura com duas operações adicionais:

- **`min()`** — retorna o menor elemento presente na pilha, sem removê-lo;
- **`max()`** — retorna o maior elemento presente na pilha, sem removê-lo.

Ambas as operações devem:

- Executar em tempo $O(1)$ (tempo constante);
- Não remover o elemento da pilha;
- Permanecer corretas após qualquer sequência de push/pop;
- Tratar corretamente valores duplicados.

2. Solução Desenvolvida

2.1 Ideia Central

A abordagem utiliza duas pilhas auxiliares que acompanham, em paralelo, a evolução do mínimo e do máximo conforme elementos são inseridos ou removidos da pilha principal.

A observação fundamental é:

A cada push, é possível calcular o novo mínimo e o novo máximo em $O(1)$, comparando o valor inserido com o topo das pilhas auxiliares.

Assim, o topo de `_min_stack` sempre guarda o menor elemento da pilha principal no estado atual, e o topo de `_max_stack` guarda o maior. A sincronização é mantida porque push e pop acontecem sempre nas três pilhas juntas.

2.2 Estrutura de Dados

A classe `MinMaxStack` mantém três listas internas:

```
class MinMaxStack:
    def __init__(self):
        self._data = [] # Pilha principal
        self._min_stack = [] # Pilha auxiliar: min corrente em cada nível
        self._max_stack = [] # Pilha auxiliar: max corrente em cada nível
```

2.3 Operação push(value)

Ao inserir um elemento, o algoritmo:

- Insere o valor na pilha principal;
- Compara com o topo de `_min_stack` e empilha o menor dos dois;
- Compara com o topo de `_max_stack` e empilha o maior dos dois.

```
def push(self, value):
    self._data.append(value)

    # Novo mínimo = min(value, mínimo anterior)
    if not self._min_stack or value <= self._min_stack[-1]:
        self._min_stack.append(value)
    else:
        self._min_stack.append(self._min_stack[-1])

    # Novo máximo = max(value, máximo anterior)
    if not self._max_stack or value >= self._max_stack[-1]:
        self._max_stack.append(value)
    else:
        self._max_stack.append(self._max_stack[-1])
```

2.4 Operação pop()

O pop remove o topo das três pilhas simultaneamente, restaurando automaticamente o min e max anteriores:

```
def pop(self):
    if self.is_empty():
        raise IndexError('pop em pilha vazia')
    self._min_stack.pop()
    self._max_stack.pop()
    return self._data.pop()
```

2.5 Operações min() e max()

Ambas simplesmente consultam o topo da pilha auxiliar correspondente:

```
def min(self):
    if self.is_empty():
        raise IndexError('min() em pilha vazia')
    return self._min_stack[-1]

def max(self):
    if self.is_empty():
        raise IndexError('max() em pilha vazia')
    return self._max_stack[-1]
```

3. Análise de Complexidade

3.1 Tabela Resumo

Operação	Tempo	Espaço	Justificativa
push(v)	O(1)	O(1)	Append à lista + append nas auxiliares
pop()	O(1)	O(1)	Pop das três pilhas simultaneamente
peek()	O(1)	O(1)	Acesso ao último índice da lista
min()	O(1) ✓	O(1)	Topo da pilha auxiliar _min_stack
max()	O(1) ✓	O(1)	Topo da pilha auxiliar _max_stack
is_empty()	O(1)	O(1)	Verificação do tamanho
size()	O(1)	O(1)	len() em lista Python

3.2 Justificativa: min() e max() em O(1)

3.2.1 O que significa O(1)?

Uma operação é O(1) — tempo constante — quando o número de passos executados não depende do tamanho n da entrada. Independentemente de haver 1 ou 1.000.000 elementos na pilha, a operação realiza sempre a mesma quantidade fixa de trabalho.

3.2.2 Por que min() e max() são O(1) nesta implementação?

As operações min() e max() reduzem-se a:

- Verificar se a pilha está vazia (`len(self._data) == 0`) — O(1) em listas Python;

- Acessar o último elemento de uma lista (`self._min_stack[-1]`) — $O(1)$ garantido pela implementação de array dinâmico do Python.

Não há nenhum laço de repetição, nenhuma chamada recursiva e nenhuma operação cujo custo cresça com n . O resultado é entregue em tempo estritamente constante.

3.2.3 Por que a abordagem ingenua seria $O(n)$?

Uma implementação sem as pilhas auxiliares precisaria varrer todos os elementos da pilha para encontrar o mínimo ou o máximo, resultando em $O(n)$. A solução apresentada pré-computa e armazena esse valor a cada push, trocando espaço de memória por velocidade de consulta.

3.2.4 Corretude com pop e valores duplicados

Cada posição das pilhas auxiliares armazena o min/max vigente no momento em que aquele elemento foi inserido. Ao executar `pop()`, o topo das três pilhas é removido conjuntamente, expondo automaticamente o min/max correto para o novo estado da pilha principal — sem nenhuma recalculação.

Valores duplicados são tratados corretamente porque os operadores `<=` e `>=` (em vez de `<` e `>`) garantem que o valor seja registrado mesmo quando igual ao atual mínimo ou máximo. Isso evita que um pop subsequente descarte um duplicado e produza um resultado errado.

4. Exemplo Passo a Passo

A tabela abaixo rastreia o estado interno para a sequência `push(3)`, `push(1)`, `push(5)`, `push(2)`, `pop()`, `pop()`, `pop()`, `pop()`:

Operação	<code>_data</code>	<code>_min_stack</code>	<code>min()</code>	<code>max()</code>
<code>push(3)</code>	[3]	[3]	3	3
<code>push(1)</code>	[3, 1]	[3, 1]	1	3
<code>push(5)</code>	[3, 1, 5]	[3, 1, 1]	1	5
<code>push(2)</code>	[3, 1, 5, 2]	[3, 1, 1, 1]	1	5
<code>pop()</code> →2	[3, 1, 5]	[3, 1, 1]	1	5
<code>pop()</code> →5	[3, 1]	[3, 1]	1	3
<code>pop()</code> →1	[3]	[3]	3	3
<code>pop()</code> →3	[]	[]	—	—

Observação: após o pop do elemento 5, o mínimo retorna a 1 imediatamente — sem varredura — porque o topo de `_min_stack` já guardava esse valor.

5. Complexidade de Espaço

A solução utiliza espaço $O(n)$ no total, onde n é o número de elementos presentes na pilha:

- `_data`: $O(n)$
- `_min_stack`: $O(n)$
- `_max_stack`: $O(n)$

O fator constante é 3 — a solução usa exatamente o triplo do espaço de uma pilha simples. Esse é o custo pela garantia de $O(1)$ para `min()` e `max()`.

Não existe solução que mantenha `min()` e `max()` em $O(1)$ sem armazenar informação adicional proporcional a n , portanto $O(n)$ de espaço é o ótimo assintótico para esta classe de problema.

6. Conclusão

A solução de pilha dupla auxiliar é a abordagem clássica e ótima para o problema. As propriedades alcançadas são:

Propriedade	Resultado
Tempo de <code>min()</code> e <code>max()</code>	$O(1)$ – constante
Tempo de <code>push()</code> e <code>pop()</code>	$O(1)$ – constante
Espaço total	$O(n)$ – ótimo para esta classe
Suporte a duplicados	Sim – operadores <code><=</code> e <code>>=</code>
Consistência pós-pop	Sim – restauração automática